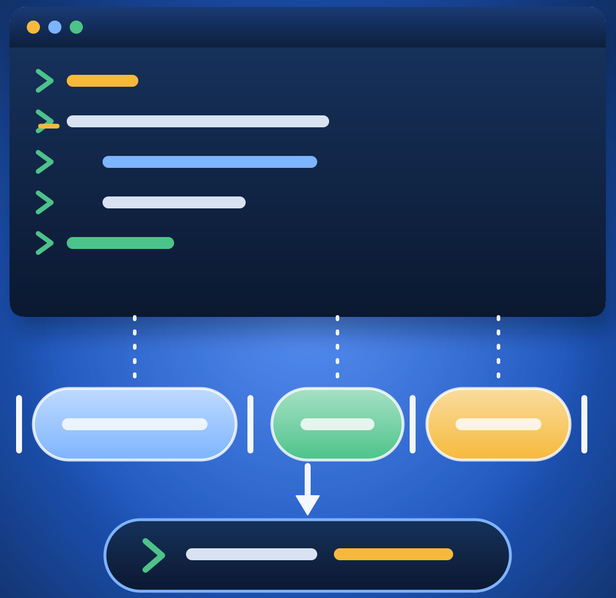
Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · LAB 7

Bash Scripting and Safe Arguments

Lab 7 guide: turn commands into a script, keep every argument whole, and report failure with an exit status.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Create and run a Bash script with arguments you understand and the right execute permission.
2. Handle spaces, names that start with a dash, and missing input without unwanted word splitting.
3. Test a reusable command, and explain its exit status and its safe handling of file names.

THIS LAB

Lab 7

First the concepts, with examples. Then the lab on the server. A personal PATH, login messages and the outbox are optional extensions, so they come last.

DISCUSSION

A file is called one file.txt. How many arguments is that?

To you it is one name. To the shell, with a space inside it, how many words is it?

CONCEPTS FIRST

Scripts, arguments and quoting

Four ideas: a script is a saved command, arguments need quotes, option parsing needs --, and every command ends with a status.

A script is a saved list of commands

show_args.sh

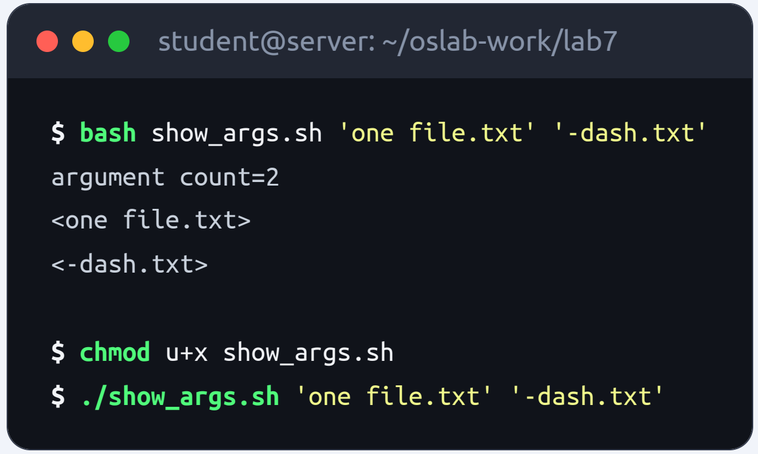
```
#!/usr/bin/env bash
printf 'argument count=%s\n' "$#"
for argument in "$@"; do
    printf '<%s>\n' "$argument"
done
```

The first line, the shebang, names the interpreter that should run the file.

$# is the number of arguments. $@ is the list of arguments. for repeats the body once for each of them.

printf with angle brackets shows exactly where each argument starts and ends.

Two ways to run it



bash script runs the file with Bash, whether or not it is executable.

./script runs it directly. That needs the execute bit and the shebang. ./ names the file in the current folder, which is not on PATH.

chmod u+x gives the owner the execute bit.

If a direct run says Permission denied, the file may lack the execute bit, or the folder may be mounted noexec. bash script is the allowed fallback.

Quotes keep an argument whole: "$@"

Quoted: "$@"

student@server: ~/oslab-work/lab7

```
$ for a in "$@"; do printf '<%s>\n' "$a"; done
<one file.txt>
```

Each argument stays one item, spaces inside it and all.

Unquoted: $@

student@server: ~/oslab-work/lab7

```
$ for a in $@; do printf '<%s>\n' "$a"; done
<one
<file.txt>
```

The shell splits on spaces. One name becomes two words, and the script looks for files that do not exist.

Quote every expansion that holds a file name: "$path", "$@", "$file".

A name that starts with a dash



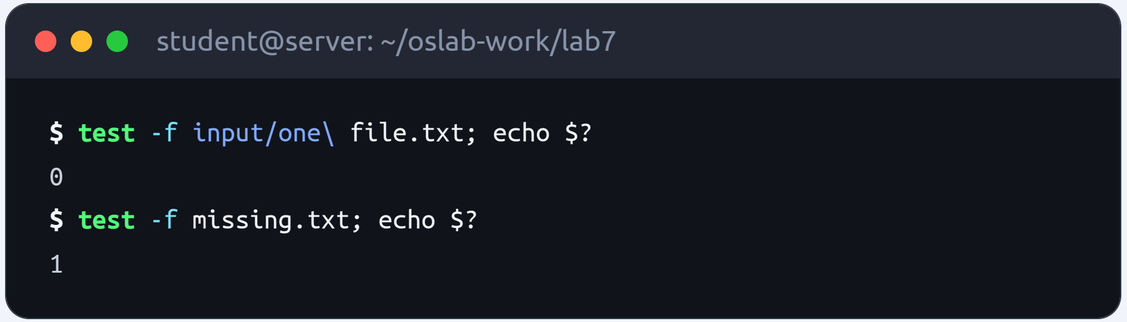
A word that starts with - looks like an option, so wc tries to read it as one.

-- tells a command that the options have ended. Everything after it is a file name.

Quoting alone does not help: the quotes are removed by the shell before wc sees the word. Writing ./-dash.txt also works.

Every command ends with an exit status

Status	Meaning
0	Success
1 and above	Some kind of failure
2	By habit, a wrong use of the command



$? holds the status of the last command. A script ends with the status you give to exit, or with the status of its last command.

LAB 7

Your own file command

Build a personal TechCorp command that inspects files. It must keep every argument whole.

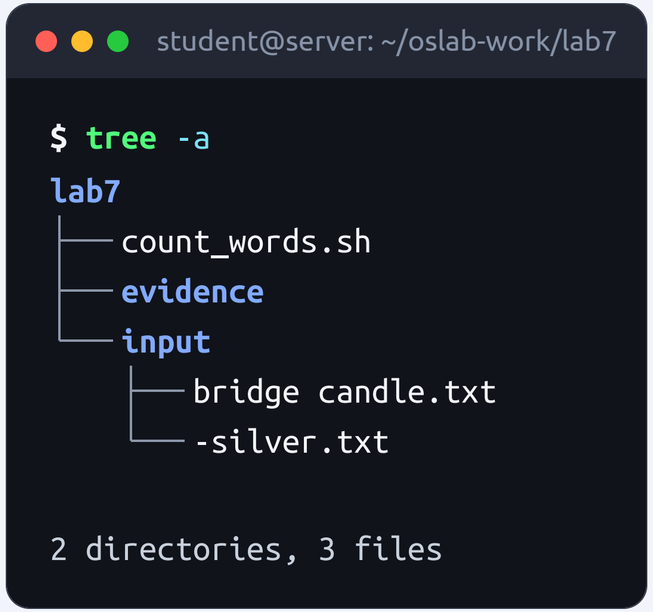
The scenario

SCENARIO

A command for your own files

Build a personal TechCorp command that inspects files in your own server neighbourhood. It must keep argument boundaries intact before any cross-user automation is considered.

Your two file names and your word counts are your own. This guide shows example values.



Inside the 120 minutes

Minutes	What happens
0 to 10	Setup
10 to 25	Guided: script basics
25 to 35	Prediction
35 to 55	Core 1: build count_words.sh
55 to 70	Core 2: test it

Minutes	What happens
70 to 85	Instructor demonstration, Plus and Challenge
85 to 100	Live checkpoint
100 to 110	Debrief
110 to 120	Submit

Six helper commands, and why you use them

Command	What it does
`oslab values lab7`	Shows your two file names. They are already in your input/ folder
`oslab predict lab7`	Saves your prediction once, before the experiment
`oslab hint lab7 1`	A hint, levels 1 to 3. Hints cost no points
`oslab check lab7`	Runs your count_words.sh on test files and shows your milestones
`oslab checkpoint lab7`	Opens when the instructor says so, near the end
`oslab debrief lab7`	After the lab: you give the right answers to your prediction questions

Before the lab, run oslab doctor and oslab prelab lab7. It takes about ten minutes.

How you work

Core, Plus, Challenge

Core is for everyone. Plus and Challenge are for students who finish early. Full marks need only Core and the checkpoint.

AI tools

Allowed in Core, Plus and Challenge, so test every suggestion with your own file names. Not allowed in the prediction and the checkpoint.

You write the script

In Core 1 you receive an empty skeleton. You write the script, and you decide how to keep the exit status.

Do not use eval, and do not run scripts you did not write. A wrong prediction costs nothing.

Setup: start the lab

student@server: ~

```
$ export OSLAB_WORKSPACE="${OSLAB_WORKSPACE:-$HOME/oslab-work}"
$ oslab start lab7
$ cd "$OSLAB_WORKSPACE/lab7"
$ mkdir -p evidence
$ oslab values lab7
```

- The first line sets OSLAB_WORKSPACE to ~/oslab-work, unless it is already set. Type it as it is.
- oslab start lab7 makes the lab folder.
- cd takes you into it, and mkdir -p makes the evidence folder.
- oslab values lab7 prints your own file names.

Your values: two file names that are only yours

The last command prints them

example: your values will be different

```
$ oslab values lab7
Your personal values for lab7 (your-account):
  file1    = bridge candle.txt
  file2    = -silver.txt
```

What to notice

- file1 has spaces in its name. file2 starts with a dash.
- Your own names are different. Write them down.
- Both files are already in your input/ folder.

The capital words are placeholders

The instruction says	With the example values, you type
`bash show_args.sh 'FILE1' 'FILE2'`	`bash show_args.sh 'bridge candle.txt' '-silver.txt'`
`wc -w -- 'input/FILE1'`	`wc -w -- 'input/bridge candle.txt'`

The instruction cannot know your names, so it writes FILE1 and FILE2 where your values belong. Never type the capital words. Type your own name in their place, between single quotes when it has a space.

Your folder

student@server: ~/oslab-work/lab7

```
$ tree -a -L 3
lab7
├── count_words.sh
├── evidence
└── input
    ├── bridge candle.txt
    └── -silver.txt

2 directories, 3 files
```

count_words.sh is an editable starter skeleton.

input/ holds your two files: one with spaces in its name, one with a leading dash. Each holds a few words.

oslab check lab7 shows TRY on every line at the start. The hidden .oslab-managed.json is the helper's. Leave it.

GUIDED

Script basics

Ten to twenty-five minutes, with the instructor. See how a script receives its arguments, and why quotes matter.

Step 1: write the argument viewer

example names: use your own

```
$ cat > show_args.sh <<'SH'
$ #!/usr/bin/env bash
$ printf 'argument count=%s\n' "$#"
$ for argument in "$@"; do
$     printf '<%s>\n' "$argument"
$ done
$ SH
$ bash show_args.sh 'bridge candle.txt' '-silver.txt'
```

cat > file <<'SH' writes everything up to the line SH into the file. This is a here-document.

The quotes around SH stop your own shell from changing $# and $@ while you type.

Expected: a count of 2, then each argument in angle brackets. "$@" keeps the spaces inside an argument.

Step 2: run it directly

example names: use your own

```
$ chmod u+x show_args.sh
$ ./show_args.sh 'bridge candle.txt' '-silver.txt'
```

The first line, #!/usr/bin/env bash, is the shebang. It chooses the interpreter for a direct run.

./ says in the current folder. The folder does not need to be in PATH.

TRY

Compare

Is the output the same as with bash show_args.sh? Why does ./ need the execute bit when bash does not?

Step 3: what the missing quotes do

example name, typed without quotes

```
$ bash show_args.sh bridge candle.txt
argument count=2
<bridge>
<candle.txt>
```

Type your own file1 name without quotes. The shell splits it at every space before the script starts.

TRY

Think

How many arguments does the script receive? Why?

Step 4: safe operands for wc

example names: use your own

```
$ wc -w -- 'input/bridge candle.txt'
$ cd input
$ wc -w -- '-silver.txt'
$ cd ..
```

-- ends the list of options for wc, so a name that starts with a dash is read as a file.

TRY

Observe

Inside input/, what would wc -w -silver.txt do without the --? Try it once. The error is intended.

PREDICT

Guess first, then try

Before you write the script. Ten minutes.

The prediction

student@server: ~/oslab-work/lab7

```
$ oslab predict lab7
```

It asks three questions and one sentence, with your own file names. Answer alone, without AI.

PREDICTION

The rules

The first answer is the one that counts. A wrong guess costs nothing. You compare it with the real result in the debrief.

CORE 1

Build count_words.sh

Twenty minutes, from minute 35 to 55. A reusable command that counts the words of the files it is given.

What count_words.sh must do

Requirements

- No arguments: print a usage message on stderr and end with a status other than 0.
- For each regular file: print one line with the file's name and its word count. The wording of the label is yours.
- For a missing or non-regular file: a message on stderr, continue with the next arguments, and a final status other than 0.
- Keep each argument in one piece, and pass file names to wc after --. Do not use eval.

YOUR TURN

You write it

Build it from the argument loop of the guided part. Different, reasonable output labels are accepted.

Step 1: look before you edit

example names: use your own

```
$ cat count_words.sh
$ cat 'input/bridge candle.txt' 'input/-silver.txt'
```

Read the skeleton and your two input files first.

Both file names are in quotes when they contain a space.

Steps 2 and 3: grow it, and keep the status

example names: use your own

```
$ nano count_words.sh
$ bash count_words.sh 'input/bridge candle.txt'
$ bash count_words.sh 'input/bridge candle.txt' 'input/-silver.txt'
```

Open the script with nano and write it. Start with one valid file, then two, and test each step before you add the error handling.

THINK

Keep the status

Keep the status in a variable that starts at 0. Change it when an input fails, and finish with exit "$status". Why is the variable better than relying on the last command?

Step 4: make it executable

example name: use your own

```
$ chmod u+x count_words.sh
$ ./count_words.sh 'input/bridge candle.txt'
$ bash count_words.sh 'input/bridge candle.txt'
```

Run it directly, then through bash, and compare the behaviour.

STUCK?

Hints

oslab hint lab7 1, then 2, then 3. AI is allowed here: test every suggestion with your own file names.

Test it

Fifteen minutes, from minute 55 to 70. Show that the command keeps names intact and reports failures.

Steps 1 and 2: a normal run, and a failure

example names: use your own

```
$ bash count_words.sh 'input/bridge candle.txt' 'input/-silver.txt' | tee evidence/normal.txt

$ {
$   bash count_words.sh missing.txt 'input/bridge candle.txt'
$   printf 'status=%s\n' "$?"
$ } > evidence/edge.txt 2>&1
$ cat evidence/edge.txt
```

Save the normal run with both files in evidence/normal.txt.

Then run it with a missing file first and a valid file second. The valid file must still be counted, and the status must not be 0.

Step 3: a real leading dash

example name: use your own

```
$ cd input
$ bash ../count_words.sh '-silver.txt' | tee ../evidence/dash.txt
$ cd ..
```

Inside input/, the name really starts with a dash, so the script must put -- before it.

The result goes into evidence/dash.txt.

What each test shows

Case	Expected	A weak script that it exposes
Two valid files	A count for each, names intact	A loop over unquoted $@
Missing, then valid	A message, a result, status above 0	A script that stops at the first error, or hides it
Leading dash, from inside input/	A count for the file	A script with no -- before the operands
Last command decides the status	Status above 0 although the last file was fine	A status taken from the last command

Direct execution can fail with Permission denied: check the execute bit, or use bash script.

Steps 4 and 5: check, then explain

The four Core milestones must show PASS: a direct run, two files, a leading dash, and errors with the status.

student@server: ~/oslab-work/lab7

```
$ oslab check lab7
```

REPORT

Write down for your report

What does each test contradict? A loop over unquoted $@, a missing --, and a status taken from the last command each fail one of the tests. Which one?

Plus: give it a folder

Give your script a folder instead of a file.

student@server: ~/oslab-work/lab7

```
$ mkdir -p input/folder
$ bash count_words.sh input/folder
```

PLUS

Answer in evidence/plus.txt

What does it print, and what is the final status? Does your test [[-f "$file"]] treat a folder like a missing file?

First watch the instructor run a script on a file name with a space, with and without quotes. Plus and Challenge are not needed for full marks.

Challenge: * or ./*

student@server: ~/oslab-work/lab7

```
$ cd input
$ bash ../show_args.sh *
$ bash ../show_args.sh ./*
$ cd ..
```

The shell replaces * by the file names before the script starts.

CHALLENGE

Answer in evidence/plus.txt

Which form is safer when a name starts with a dash, and why?

LIVE CHECKPOINT

Show what you can do alone

Fifteen minutes near the end of the lab. Worth 3 of the 10 points.

Why is there a checkpoint?

It shows what you can do alone

For most of the lab you may use hints and AI tools. The checkpoint is the one moment without them.

The file name is new

The questions use a new, awkward file name, made for you at that moment. An answer copied from a neighbour does not fit.

The instructor sees who needs help

Your answers arrive at once, so a common mistake can be explained before the next lab.

It is fair

Everyone gets the same kind of task, with their own file name.

What happens, step by step

1. The instructor opens it
Close your AI tools

→

2. oslab checkpoint lab7
Answer the four questions

→

3. The file task
Create the new file, run your script

→

4. oslab check lab7
A fifth milestone must pass

Before it opens

The command says that the checkpoint is not open yet. Wait for the instructor.

While you work

Use your terminal and what you learned in this lab. No AI tools, and no help from a neighbour.

When you finish

oslab check lab7 shows whether the fifth milestone, checkpoint, passes. You can fix your script and check again.

What the checkpoint asks

Part	What you do
Four questions	They use a new, awkward file name made for you at this moment. Answer them in the terminal.
The file task	Create the new file the command prints, run your own count_words.sh on it, and save the output in checkpoint.txt.
The check	Run oslab check lab7. The fifth milestone, checkpoint, must show PASS.

The same script must handle the new name. This is why Core 2 tested spaces and a leading dash.

DEBRIEF AND SUBMIT

Look back, then hand in

Ten minutes of debrief, ten to submit.

Debrief: look at your prediction again

student@server: ~/oslab-work/lab7

```
$ cat "$OSLAB_WORKSPACE/.records/lab7-predict.json"
$ oslab debrief lab7
```

oslab saved your prediction in a hidden folder, .records. It stays as it was, so the comparison shows which answers you confirmed and which you corrected.

DISCUSS

Class discussion

Your script worked on the files you tested. Which file name could still break it?

Submit 1: your repository and the copy

student@server: ~

```
$ SUBMISSION_REPO="$HOME/os-gic-YOUR_ID/os-lab-YOUR_ID"
$ mkdir -p "$SUBMISSION_REPO/lab7/evidence"
$ cd "$OSLAB_WORKSPACE/lab7"
$ cp -- count_words.sh "$SUBMISSION_REPO/lab7/"
$ cp -- evidence/normal.txt evidence/edge.txt "$SUBMISSION_REPO/lab7/evidence/"
```

Replace YOUR_ID with your own ID. The orange text is the part you change. Use the absolute path of your existing course repository, never a neighbour's.

Example: with the ID 0123, the first line becomes SUBMISSION_REPO="$HOME/os-gic-0123/os-lab-0123", and echo "$SUBMISSION_REPO" prints /home/student/os-gic-0123/os-lab-0123.

If you did Plus or Challenge, copy evidence/plus.txt the same way. The course website checks that these exact names are there.

Submit 2: the final tree and Git

student@server: ~/os-gic-YOUR_ID/os-lab-YOUR_ID

```
$ tree
lab7
├── count_words.sh
├── evidence
│   ├── edge.txt
│   └── normal.txt
└── README.md

1 directory, 4 files
```

1. Write README.md from the lab's template, in your own words.
2. The prediction must have been written before you ran anything. Copying it into the README only records it.
3. git status --short, then add only your lab files, then commit and push.
4. No passwords, personal shell files, generated binaries or disk images.

Submit 3: commit and push

student@server: ~

```
$ cd ~/os-gic-YOUR_ID
$ git pull
$ git add os-lab-YOUR_ID/lab7
$ git commit -m "Lab 7"
$ git push
```

git pull comes first, in case you pushed from your own computer since.

Replace YOUR_ID with your own ID, as before.

How it is marked: 10 points

Evidence for the lab objectives	Points
Script: oslab check lab7 passes the four Core milestones (direct run, two files, leading dash, errors and status)	2
Tests: normal.txt and edge.txt show the runs, and the README says what each test contradicts	2
Prediction saved in time, and corrected with oslab debrief lab7	2
Live checkpoint: the answers and your script counting the new file	3
Complete submission: the required files in your repository, on time	1

Every row is marked from your oslab records and the files in your repository, so run oslab check lab7 until it shows what you expect. Your prediction earns its first point for being made, not for being right. Plus and Challenge are not needed for full marks.

Common mistakes

Mistake	What you see	Fix
for a in $@ without quotes	A name with a space split into two words	Write "$@"
No -- before operands	wc: invalid option	Put -- before the file names
Exit 0 after a failed input	A script that always looks successful	Keep a status variable, exit with it
Running ./script without chmod	Permission denied	chmod u+x, or use bash script
Typing FILE1, or a spaced name without quotes	No such file, or two arguments	Use your own name, in single quotes

BEYOND THE CORE

PATH, login messages and the outbox

Optional extensions. Cross-user work needs a narrow folder the instructor prepared.

Lab 7

Your own commands on PATH

student@server: ~/oslab-work/lab7

```
$ mkdir -p ~/bin
$ cp count_words.sh ~/bin/count_words
$ export PATH="$HOME/bin:$PATH"
$ command -v count_words
/home/student/bin/count_words
```

PATH is the list of folders the shell searches for a command name. Add your own folder in front, and the script runs without ./.

export PATH=... lasts for this shell only. Put it in ~/.bashrc to keep it, and read that file again with source ~/.bashrc.

Login messages, outbox and the mailbox idea

LOGIN

A message when you log in

A line in ~/.bash_profile or ~/.bashrc can print a notice. Preview it with a plain command before you rely on it.

OUTBOX

An owned outbox

A folder you own where a script leaves messages. Only you write to it, so permissions stay simple.

MAILBOX

A cross-user mailbox

Needs a narrow shared folder the instructor prepared, with write permission only for the people who need it. Never open your own home.

A program that runs with its owner's rights (setuid) is a concept for a prepared virtual machine. It is not part of this lab.

Key takeaways

01

A script is saved commands

A shebang names the interpreter, and chmod u+x lets you run the file directly.

02

Quote what you expand

"$@" keeps each argument whole. Unquoted, a space splits it into two words.

03

-- ends the options

Use it before file names, so a leading dash cannot be read as an option.

04

The status is the message

Exit non-zero when any input failed, even if you carried on with the rest.

Next session

COMING UP

Lab 8: secure scripts, races and file locking

Two scripts touch one file at once. See what goes wrong, and fix it with a lock.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 2, Operating-System Structures (the command interpreter), and the Bash manual.